

.NET 简历项目面试 Q&A

口径建议：可以说 5 年 .NET 经验，其中包含 2021 年起实习参与项目。表达重点放在“参与核心业务、负责模块或子系统落地、能独立排查问题”，不要说“主导整个系统架构”。

一、项目介绍与履历口径

Q1: 你怎么介绍自己的项目经历？

回答结构：

1. 年限：5 年 .NET 经验，包含 2021 年实习经历。
2. 行业：医疗信息化、政企平台、ERP、第三方接口、设备采集。
3. 职责：模块开发、接口设计、表设计、联调、上线排查。
4. 技术：ASP.NET Core、EF Core、Dapper、Redis、RabbitMQ、SQL Server、MongoDB、Hangfire。

可直接口述：

我有 5 年 .NET 开发经验，其中 2021 年开始实习参与医疗信息化项目，后续转正后持续负责业务模块开发。做过的项目主要有 HDIS 血液透析系统、CNRDS 国家平台上报、ETL 医疗数据集成、政企专家考试、电子发票网关、金融保函、ERP 和设备采集客户端。我的职责主要是负责模块或子系统的接口设计、数据库设计、核心代码开发、前后端联调和上线问题排查，不是整个项目的架构决策人。

Q2: 你为什么把实习也算进 5 年经验？

推荐回答：

我从 2021 年开始实习参与实际项目，不是单纯学习性质，实习阶段也参与了 HDIS 医疗系统的业务开发、接口联调和数据处理。后续转正后在同一业务方向继续深入，所以简历上按 5 年含实习经验来写。如果严格按正式工作年限算会少一些，但项目连续性和实际参与时间是从 2021 年开始的。

不要这样说：

- 我就是 5 年正式工作经验。
 - 实习和正式没区别。
 - 整个项目主要都是我负责的。
-

Q3: 面试官问“你独立负责过什么”怎么回答？

推荐回答：

我独立负责过边界比较清晰的子系统或模块，比如 CMS 存量资产系统、电子发票网关里的开票记录和第三方适配模块、HDIS 里的 CNRDS 上报和库存相关功能。这里的独立是指我能完成需求理解、接口设计、表设计、编码、联调和上线问题处理，不是说整个项目的技术选型和架构都是我一个人决定。

Q4: 2 分钟介绍一个你最熟悉的项目

模板：

我比较熟的是 xxx 项目，主要解决 xxx 业务问题。
系统使用 ASP.NET Core / EF Core / Dapper，数据库是 SQL Server / MySQL，
中间件用到了 Redis / RabbitMQ / Hangfire。
我主要负责 xxx 模块，包括接口设计、表结构设计、核心功能开发和上线问题排查。
其中有三个比较典型的问题：
第一，xxx，用 xxx 解决；
第二，xxx，用 xxx 保证；
第三，xxx，通过 xxx 优化。

示例：

我比较熟的是 HDIS 血液透析信息系统，主要服务血透中心的患者建档、排班、处方、治疗记录、费用结算、库存和 CNRDS 上报。后端使用 ASP.NET Core、EF Core、Dapper、Redis、RabbitMQ 和 SQL Server。我主要负责患者、排班、库存、CNRDS 上报等模块的接口开发和问题排查。比较典型的问题有三个：库存扣减用乐观锁和流水保证并发安全；CNRDS 上报通过字段映射、校验、日志和补报保证数据质量；报表慢查询通过联合索引、SQL 调整和缓存优化。

二、政企平台与第三方接口

Q5: 专家培训考试系统如何处理开考高峰登录？

流程图：

```
登录/验证码请求
|
v
解析手机号 + IP + 设备标识
|
v
Redis 滑动窗口计数
|
+--> 超过阈值：拒绝/提示稍后再试
|
v
账号密码/验证码校验
|
v
生成 Token + 记录登录日志
```

口述答案：

开考高峰主要压力在集中登录、验证码和反复刷新。我会按手机号、IP、设备标识做多维度限流，Redis 里用滑动窗口记录单位时间内请求次数，超过阈值就拦截。这样比固定窗口更平滑，能避免窗口边界突刺。登录成功后再生成 Token，并记录登录日志，方便后续排查异常登录。

追问：Redis 挂了怎么办？

限流不能完全把系统可用性绑死在 Redis 上。Redis 不可用时可以短时间降级为本地内存限流或更保守的默认策略，同时记录告警。核心业务仍然要保护数据库和登录接口，避免无限放开。

Q6: 考试交卷如何防重复提交？

流程图：

```
交卷请求
|
v
生成幂等 Key: exam:{examId}:user:{userId}
|
v
Redis SETNX(key, requestId, ttl)
|
+--> false: 返回已提交/处理中
|
v
开启事务
|
v
保存答卷 + 计算成绩 + 更新考试状态
|
v
数据库唯一约束兜底
```

口述答案：

交卷是典型的幂等场景。前端可能重复点击，网络超时后也可能重试。我会用考生 ID + 试卷 ID 作为幂等 Key，先通过 Redis SETNX 判断是否已经提交或正在提交。真正写库时，数据库也会对考生和试卷建立唯一约束兜底。Redis 负责挡住大部分重复请求，数据库负责最终一致性。

Q7: 视频转码为什么要异步？怎么设计？

流程图：

```
前端上传视频
|
v
保存文件信息(status=UPLOADED)
|
v
创建转码任务(status=WAITING)
|
v
后台任务调用 FFmpeg
|
+--> 成功: status=SUCCESS, 保存播放地址
|
+--> 失败: status=FAILED, 记录失败原因
|
v
WebSocket/轮询通知前端
```

口述答案：

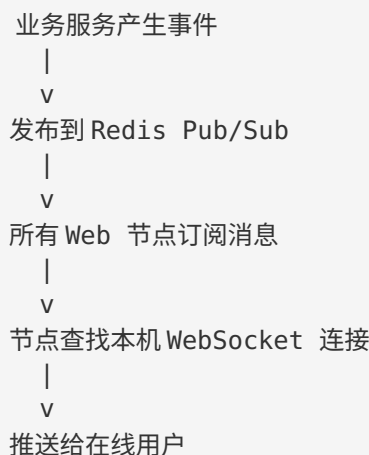
视频转码耗时比较长，如果在上传接口里同步处理，容易导致 HTTP 超时，也会占用 Web 线程。我的设计是上传接口只负责保存文件和创建任务，快速返回；后台任务再调用 FFmpeg 转码，更新任务状态。前端通过 WebSocket 或轮询查看进度。失败的任务保留失败原因，可以手动重试。

追问：多实例下如何避免重复转码？

可以用任务状态抢占或 Redis 锁。比如把任务从 WAITING 更新为 PROCESSING 时带原状态条件，只有更新成功的节点才能处理。这样即使多个节点同时扫描任务，也只有一个节点能执行。

Q8: WebSocket 多实例部署怎么推送消息？

流程图：

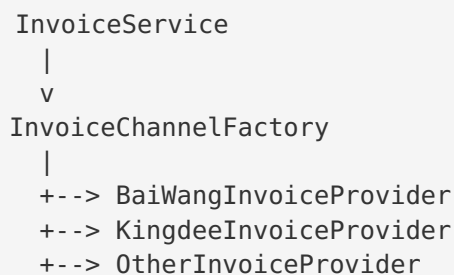


口述答案：

WebSocket 连接是挂在具体节点上的，多实例部署时不能只在本机内存里发消息。我会用 Redis Pub/Sub 做跨节点消息总线，业务事件发布到 Redis，各个节点都订阅。收到消息后，每个节点只给本机连接的用户推送。这样用户不管连接到哪个实例，都能收到转码进度或考试状态。

Q9: 电子发票网关为什么用工厂/策略模式？

结构图：

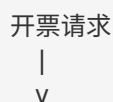


口述答案：

不同税务厂商的报文格式、签名规则、状态码和回调字段差异比较大。如果全部写在一个 Service 里，会出现大量 if/else，维护成本很高。我把内部开票流程抽象成统一接口，不同厂商用不同 Provider 实现。业务层只关心统一入参和统一结果，新增渠道时主要新增实现类和配置，减少对核心流程的影响。

Q10: 电子发票如何防重复开票？

流程图：



```

校验业务流水号
|
v
Redis SETNX(serialNo)
|
+--> 失败：查询已有开票记录并返回
|
v
写入本地开票记录(status=PROCESSING)
|
v
调用第三方开票
|
v
更新 SUCCESS / FAILED / UNKNOWN

```

口述答案：

电子发票最怕重复开票，所以我会先保证业务流水号唯一。请求进入后先用 Redis SETNX 做幂等，如果失败就查询本地已有记录返回。调用第三方前先落本地记录，状态为处理中；第三方返回后再更新状态。如果接口超时，不能直接再次开票，而是标记 UNKNOWN，通过定时任务查询第三方状态。

Q11: 电子发票回调如何保证状态正确？

状态机：

```

INIT
|
v
PROCESSING
|
+--> SUCCESS
|
+--> FAILED
|
+--> UNKNOWN --查询补偿--> SUCCESS / FAILED

```

口述答案：

回调首先要验签，确认是合法第三方请求。然后根据外部流水号查本地开票记录。更新状态时必须按状态机流转，比如 SUCCESS 不能再被 FAILED 覆盖，PROCESSING 或 UNKNOWN 才允许更新为最终状态。回调可能重复，所以接口要幂等。长时间没有回调的数据通过 Hangfire 定时任务主动查询第三方状态。

Q12: 第三方接口对接你一般关注什么？

回答要点：

- 超时：连接超时和读取超时要分开配置。
- 幂等：写操作必须有业务流水号。
- 签名：时间戳 + nonce + 摘要，防篡改和重放。
- 日志：保存请求、响应、耗时、状态码、错误原因。
- 补偿：失败数据可查询、可重试、可人工处理。

口述答案：

第三方接口不能只看调通，重点是异常场景。比如超时后到底成功没成功、回调重复怎么办、状态码怎么映射、签名失败怎么排查。我一般会先定义本地状态机和业务流水号，所有请求响应都落日志。写操作不盲目重试，优先查状态；查询类接口可以做有限次数重试。

Q13: 金融保函系统的安全措施有哪些？

口述答案：

金融保函涉及企业和承保数据，我主要从传输安全、报文完整性和审计追踪几个方面处理。传输层走 HTTPS；业务报文使用 SM4 加密保护敏感内容；请求参数增加摘要签名，防止被篡改；每次请求和回调都记录流水号、机构编码、状态、耗时和异常原因，方便后续审计和排查。

三、医疗信息化项目

Q14: HDIS 血液透析系统你负责哪些模块？

口述答案：

HDIS 主要覆盖患者建档、透析排班、处方、治疗记录、费用结算、库存管理和 CNRDS 上报。我参与过患者信息、排班、处方、费用、库存和上报相关模块，主要负责后端接口、表结构、业务校验、前后端联调和线上问题排查。医疗系统的特点是数据口径复杂、历史数据多、外部系统多，所以对校验、日志和可追溯性要求比较高。

Q15: 多租户数据隔离怎么做？

流程图：

```
graph TD
    A[用户登录] --> B[获取 tenantId / userId / role]
    B --> C[写入登录上下文 / Token]
    C --> D[接口请求进入后解析上下文]
    D --> E[查询自动带 tenant_id 条件]
    E --> F[新增数据写入 tenant_id]
```

口述答案：

我们这种医疗系统通常是共享库共享表，通过 tenant_id 做逻辑隔离。用户登录后会拿到 tenantId，后端查询时通过上下文或过滤器自动带上租户条件，新增数据也写入 tenantId。缓存 Key、MQ 消息和后台任务参数也要带 tenantId，否则异步场景容易串数据。

风险点：

- 手写 SQL 漏 tenant_id。
- 报表跨表查询绕过过滤器。

- 缓存 Key 没带 tenantId。
- 后台任务没有租户上下文。

Q16: RBAC 权限体系如何设计？

模型图：

```
graph TD
    User --> UserRole
    UserRole --> Role
    Role --> RolePermission
    RolePermission --> MenuButtonApi["Menu / Button / Api"]
```

口述答案：

RBAC 核心是用户、角色、权限。用户关联角色，角色关联菜单、按钮和接口权限。前端根据菜单和按钮权限控制显示，后端根据接口权限做最终拦截。关键点是不能只依赖前端隐藏按钮，敏感接口必须要在后端校验。

Q17: CNRDS 上报具体做了什么？

流程图：

```
graph TD
    A[业务表数据] --> B[抽取患者/诊断/治疗/检查/用药/透析记录]
    B --> C[字段映射 + 编码转换]
    C --> D[必填/范围/字典/关联校验]
    D --> E["+-> 失败：记录原因，等待修正"]
    E --> F[生成上报报文]
    F --> G[调用国家平台接口]
    G --> H[记录批次/状态/失败原因]
```

口述答案：

CNRDS 上报难点不是单纯调用接口，而是数据质量。需要从患者、诊断、治疗方案、用药、实验室检查、透析记录等表里抽取数据，做字段映射、编码转换和校验。校验不通过的数据不会直接上报，而是记录失败原因。上报后按批次记录状态，支持失败重试和手动补报。

Q18: CNRDS 如果国家平台不可用怎么办？

口述答案：

本地业务不能因为国家平台不可用而阻塞。我的处理是先把待上报数据落库，状态标记为 WAITING 或 FAILED。Hangfire 定时任务按批次重试，超过重试次数后记录失败原因，给运维或业务人员处理。这样保证本地业务正常运行，同时让上报具备可补偿能力。

Q19: 医疗 ETL 数据同步怎么设计？

流程图：

```
graph TD
    A[HIS / LIS / 设备系统] --> B[接口 / 视图 / 文件 / 串口]
    B --> C[原始数据表 raw_xxx]
    C --> D[清洗转换]
    D --> E[业务表]
    E --> F[页面查询 / 上报使用]
```

清洗转换

- + -> 单位转换
- + -> 编码映射
- + -> 必填校验
- + -> 去重

口述答案：

ETL 我会分为接入、落原始表、清洗转换、写业务表几个步骤。原始数据先保存，方便追溯。清洗阶段处理单位转换、编码映射、缺失值校验和重复数据过滤。任务用 Hangfire 批处理，失败记录原因，后续可以重跑。医疗数据不能只追求同步成功，还要能解释失败原因。

Q20: IOT 医疗设备采集系统怎么做？

口述答案：

设备采集主要是按厂商协议解析数据。比如血压计、体重秤可能走串口、Modbus 或 TCP。客户端读取原始数据后，根据协议解析出收缩压、舒张压、体重等字段，再做单位转换和合法性校验，然后上传到业务系统。设备在线状态通过 TCP 或心跳检测，异常断开要记录日志并提示现场人员处理。

四、ERP 与业务系统

Q21: ERP 订单状态流转怎么设计？

状态示例：

```
DRAFT 草稿
|
v
AUDITING 审核中
|
+-> REJECTED 已驳回
|
v
APPROVED 已审核
|
v
PARTIAL_IN 部分入库 / PARTIAL_OUT 部分出库
|
v
FINISHED 完成
```

口述答案：

ERP 单据状态要用状态机思路，不能随便改状态。比如采购单从草稿到审核中，再到已审核，后续根据入库情况变成部分入库或完成。每次状态流转都要校验当前状态、操作权限和业务数据，防止重复审核、越级流转或已完成单据被修改。

Q22: ERP 库存扣减如何保证并发安全？

SQL 思路：

```
UPDATE stock
SET qty = qty - @qty,
    version = version + 1
WHERE id = @id
AND qty >= @qty
AND version = @version;
```

口述答案：

库存并发不能只靠 C# lock，因为多实例部署下锁不住数据库。我会用数据库条件更新和乐观锁。更新时带库存数量和 version 条件，影响行数为 1 才表示扣减成功。扣减成功后写库存流水，记录业务单号、批号、数量、操作人。接口层再用业务流水号做幂等，防止重复提交。

Q23: ERP 多维库存为什么复杂？

口述答案：

ERP 库存不是简单的商品数量，通常要按 SKU、仓库、库位、批号、效期、序列号、货主等维度区分。同一个 SKU 在不同批号或不同库位上是不同库存。设计表时要明确库存维度，并对关键维度建立唯一约束，避免同一份库存被拆成多条不一致记录。

Q24: BOM 和 MRP 你怎么理解?

简图:

```
成品 A
|
+--> 物料 B x 2
|
+--> 半成品 C x 1
      |
      +--> 物料 D x 3
```

口述答案:

BOM 是产品结构清单, 描述生产一个成品需要哪些物料和数量。MRP 是根据订单需求、库存、在途采购和生产计划计算物料缺口。实现时要注意 BOM 层级展开、损耗率、替代料、库存可用量和已占用量。我的参与主要是业务功能开发和数据处理, 不会把自己说成生产计划算法的负责人。

Q25: 电商平台对接怎么处理不同平台差异?

结构图:

```
ERP OrderService
|
v
PlatformAdapter
|
+--> TaobaoAdapter
+--> JdAdapter
+--> PddAdapter
```

口述答案:

不同平台的订单字段、SKU 编码、状态和接口限制都不一样。我会抽一层适配器, 把平台订单转换成内部统一订单模型。同步时要处理分页、时间窗口、漏单补偿和接口限流。状态同步也要做映射, 比如平台已发货、已完成, 对应 ERP 内部的出库或完成状态。

五、数据库与性能优化

Q26: 慢 SQL 你怎么排查?

流程图:

```
接口慢
|
v
看接口分段耗时
|
v
定位具体 SQL
|
v
查看执行计划
```

```
|
+--> 是否全表扫描
+--> 是否命中索引
+--> 是否 filesort / temporary
+--> 扫描行数是否过大
|
v
调整 SQL / 索引 / 缓存 / 分页
```

口述答案:

我会先确认慢在接口、远程调用还是数据库。定位到 SQL 后看执行计划，重点看 type、key、rows、Extra。常见优化包括补联合索引、减少 select *、调整 where 条件顺序、避免在索引列上使用函数、减少无效 join、统计类接口加缓存。优化后要对比执行计划和实际耗时。

Q27: 报表查询从 1 秒优化到几百毫秒，怎么做？

口述答案:

报表查询通常慢在数据量大、关联多、排序多。我会先找最慢 SQL，看是否全表扫描和 filesort。对高频查询条件，比如 tenant_id、patient_id、date、status 建联合索引。统计类结果如果允许短暂延迟，可以用 Redis 做短 TTL 缓存。还有些复杂报表可以提前汇总，避免每次实时扫明细表。

Q28: EF Core 和 Dapper 怎么选？

对比:

场景	选择
普通 CRUD	EF Core
聚合根/实体关系	EF Core
复杂报表 SQL	Dapper
高性能批量查询	Dapper
需要强类型追踪	EF Core

口述答案:

EF Core 开发效率高，适合普通 CRUD 和实体关系明确的业务。Dapper 更轻量，适合复杂报表、手写 SQL 和对性能比较敏感的查询。实际项目里可以混用，业务写操作和常规查询用 EF Core，复杂统计和性能热点用 Dapper。

Q29: SQL Server / MySQL 索引设计注意什么？

口述答案:

索引要围绕高频查询设计，不是越多越好。联合索引要考虑等值条件、范围条件和排序字段。比如多租户系统常见条件是 tenant_id + business_date + status，可以按查询频率设计联合索引。写入频繁的表要控制索引数量，因为每次 insert/update 都要维护索引。

Q30: MongoDB 为什么适合保存第三方响应？

口述答案:

第三方接口响应字段可能变化，MongoDB 文档结构灵活，适合保存原始请求和响应。它也支持 TTL 索引，可以自动清理临时缓存或过期日志。对于电子发票这种要追踪第三方报文、又不想频繁改关系表字段的场景比较合适。

六、并发、缓存、消息与任务

Q31: Redis 分布式锁使用要注意什么？

口述答案：

Redis 锁一定要设置过期时间，避免服务挂掉导致死锁。加锁和设置过期时间要保证原子性，比如 SET key value NX EX。释放锁时要校验 value，避免删掉别人的锁。真正关键的数据一致性不能只靠 Redis 锁，还要有数据库唯一约束、状态机或乐观锁兜底。

Q32: RabbitMQ 如何保证消息可靠？

流程图：

```
生产者发送
|
v
Confirm 确认到达 Broker
|
v
队列/消息持久化
|
v
消费者处理业务
|
+--> 成功：ack
|
+--> 失败：retry / dead letter
```

口述答案：

消息可靠要分生产端、Broker 和消费端。生产端开启 confirm，确认消息到达 Broker；队列和消息设置持久化；消费端手动 ack，业务处理成功后再 ack。重复消费不能完全避免，所以消费端必须做幂等，比如业务流水号唯一索引或 Redis SETNX。

Q33: Hangfire 后台任务如何设计成可重入？

口述答案：

后台任务可能失败、重试或重复执行，所以要设计成可重入。每条数据要有状态，比如 WAITING、PROCESSING、SUCCESS、FAILED。任务每次只处理 WAITING 或 FAILED 的数据，处理成功更新 SUCCESS。PROCESSING 超过一定时间可以重置，避免服务中断后任务卡死。已经 SUCCESS 的数据不重复处理。

Q34: Redis 缓存和数据库如何保持一致？

口述答案：

常见做法是 Cache Aside。读的时候先查缓存，没命中再查数据库并写缓存；写的时候先更新数据库，再删除缓存。不是先更新缓存，因为并发下容易产生脏数据。对于报表类缓存可以设置较短 TTL，允许短时间不一致；对于强一致业务不能依赖缓存做最终判断。

Q35: Redis 挂了系统怎么办？

口述答案：

要看 Redis 在这个场景里是缓存还是强依赖。如果只是查询缓存，Redis 挂了可以降级走数据库，但要限流，避免数据库被打爆。如果是分布式锁、限流、幂等等强依赖场景，就要保守处理，比如拒绝高风险操作或切换本地限流，并尽快告警恢复。核心数据一致性必须有数据库兜底。

七、ASP.NET Core 与工程实践

Q36: ASP.NET Core 请求管道怎么理解？

简图：

```
Request
|
v
Middleware 1
|
v
Authentication
|
v
Authorization
|
v
Endpoint / Controller
|
v
Response
```

口述答案：

ASP.NET Core 请求会按中间件顺序执行。常见中间件有异常处理、静态文件、路由、认证、授权和 endpoint。顺序很重要，比如认证要在授权之前。我们也可以自定义 Middleware 记录请求耗时、traceId、请求路径和异常信息，用于线上排查。

Q37: 你们如何记录接口和 SQL 耗时？

口述答案：

接口层可以用 Middleware 记录请求开始和结束时间，输出路径、状态码、耗时和 traceId。EF Core 可以通过 DbCommandInterceptor 拦截 SQL 执行，记录 SQL、参数和耗时。Dapper 或原生 SQL 可以通过封装 DbConnection/DbCommand 来记录。日志统一写到 Serilog，方便按 traceId 串起来看。

Q38: 统一异常处理怎么做？

口述答案：

ASP.NET Core 可以通过中间件或过滤器做统一异常处理。业务层抛出明确的业务异常，比如参数错误、状态不允许、权限不足；全局异常处理中转换成统一响应格式。系统异常要记录完整堆栈，但返回给前端的信息要简洁，避免暴露内部实现。

Q39: 接口设计如何保持一致？

口述答案：

接口返回结构要统一，比如 code、message、data、traceId。分页参数和分页返回也要统一。错误码要有规范，不要每个接口随便写。接口文档要及时更新，前后端联调时约定字段命名、时间格式和空值处理。这样可以减少联调成本。

Q40: DevOps 你具体做过什么？

口述答案：

我参与过 Jenkins 构建发布、Docker 镜像构建、Nginx 配置、Linux 日志排查等工作。K8s、Helm、Harbor 这类更多是参与部署和联调，不会说自己负责整套平台建设。面试时我会重点讲自己实际做过的构建、配置、日志定位和发布问题处理。

八、线上排查与场景题

Q41: 线上接口突然变慢，你怎么排查？

流程图：

```
确认现象
|
+--> 单接口慢还是全站慢
+--> 所有用户慢还是某租户慢
|
v
看应用资源
|
+--> CPU / 内存 / GC / 线程池
|
v
看依赖
|
+--> SQL / Redis / MQ / 第三方接口
|
v
看日志 traceId
|
v
限流/降级/回滚/修复
```

口述答案：

先确认影响范围，是单接口还是全站，是所有租户还是某个租户。然后看应用资源和日志，定位慢在 Controller、Service、SQL、Redis 还是第三方接口。如果是生产故障，先止血，比如限流、降级、回滚最近发布，再做根因修复。

Q42: 前端说接口 500，你怎么查？

口述答案：

先让前端提供请求路径、参数、时间点和 traceId。后端按 traceId 查日志，看异常堆栈。如果没有 traceId，就按时间和用户定位。确认是参数问题、权限问题、业务状态问题还是系统异常。修复后要补充校验或错误提示，避免同类问题再次出现。

Q43: 设计一个医院检查结果同步子系统

设计图：

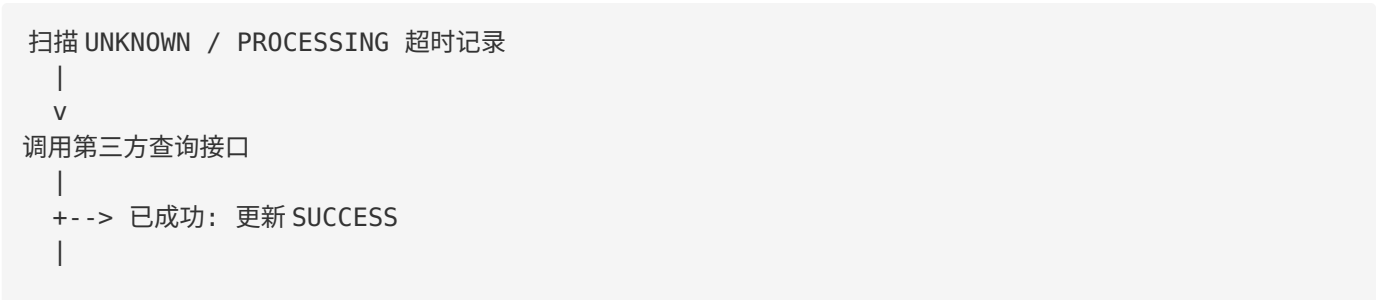


回答步骤：

1. 先确定对接方式：接口、数据库视图、文件还是消息。
2. 原始数据先落 raw 表，方便追溯。
3. 清洗层做字段映射、单位转换、字典转换、必填校验。
4. 通过外部流水号 + 患者 ID + 检查时间做去重。
5. 使用 Hangfire 定时同步，失败记录原因，支持补偿。
6. 查询侧按 patient_id、exam_time、item_code 建索引。

Q44: 设计一个电子发票补偿任务

流程图：



```
+--> 已失败：更新 FAILED
|
+--> 仍未知：retry_count + 1
|
v
超过重试上限：标记人工处理
```

口述答案：

补偿任务只处理超时未完成的数据，比如 UNKNOWN 或 PROCESSING 超过一定时间的记录。任务调用第三方查询接口确认真实状态，再更新本地状态。任务要限制批次大小和重试次数，避免第三方接口压力过大。超过上限后标记人工处理，并保留完整日志。

Q45: 设计一个库存盘点调整流程

流程图：

```
创建盘点单
|
v
冻结盘点范围库存
|
v
录入实盘数量
|
v
生成差异
|
v
审核盘点单
|
v
生成库存调整流水
|
v
更新库存数量
```

口述答案：

盘点不能直接改库存，要通过盘点单和库存流水。创建盘点单后确定盘点范围，必要时冻结库存，避免盘点期间继续出入库影响结果。录入实盘数量后生成差异，审核通过再生成调整流水并更新库存。这样每次库存变化都有来源和审计记录。

Q46: 如果面试官质疑你技术栈太广，怎么回答？

推荐回答：

我的主线是 .NET 后端，C#、ASP.NET Core、EF Core、Dapper、SQL 和 Redis 是核心能力。WinForms、Vue、Docker、MQ、MongoDB 这些是围绕项目交付积累的，不是每个都说自己很深。面试时我会优先讲自己做过的后端模块、数据一致性、性能优化和接口对接，其他技术作为协作和落地能力补充。

Q47: 未来想提升什么?

推荐回答:

我想继续加深 .NET 后端和分布式系统能力，重点是微服务治理、可观测性、消息可靠性、数据库性能优化和云原生部署。业务上希望继续做医疗、政企、ERP 这类有复杂业务规则的系统，因为这类项目能锻炼数据建模、稳定性和问题排查能力。

九、快速索引

方向	对应问题
项目介绍	Q1 / Q4
5 年含实习口径	Q2
独立负责怎么说	Q3
登录限流	Q5
交卷幂等	Q6
视频转码	Q7
WebSocket 多实例	Q8
工厂/策略模式	Q9
电子发票幂等	Q10
发票回调状态机	Q11
第三方接口	Q12
金融安全	Q13
HDIS	Q14
多租户	Q15
RBAC	Q16
CNRDS	Q17 / Q18
ETL	Q19
设备采集	Q20
ERP 状态机	Q21
库存并发	Q22 / Q23 / Q45
BOM/MRP	Q24
电商对接	Q25
慢 SQL	Q26 / Q27
EF Core vs Dapper	Q28
索引设计	Q29
MongoDB	Q30
Redis 锁/缓存	Q31 / Q34 / Q35
RabbitMQ	Q32
Hangfire	Q33
ASP.NET Core	Q36 / Q38 / Q39
耗时监控	Q37
DevOps	Q40
线上排查	Q41 / Q42
场景设计	Q43 / Q44 / Q45
技术广度质疑	Q46

方向
发展方向

对应问题
Q47